Challenge 2:

Karim Negm - 11063520

Exercise Question 1:

Compute the total energy consumed by the two battery-powered devices over a period of 24 hours in both cases when using COAP (a) and MQTT (b), using each in its most efficient configuration energy wise.

ANSWER:

(a) Total energy consumed by:
   a. Sensor: 6.39 mJ
   b. Valve: 122.60 mJ
   c. Total = 128.99 mJ

(b) Total energy consumed by:
   a. Sensor: 7.88 mJ
   b. Valve: 124.38 mJ
   c. Total = 132.25 mJ

Answer motivation:

- **(a) Using CoAP:** The Valve observes the Sensor using CoAP GET+OBSERVE, and the Sensor sends Non-confirmable (NON) updates.
- **(b) Using MQTT:** Using QoS 0 for sensor publications and assuming a Keep Alive interval long enough (>5 min) to avoid the need for MQTT PING messages.

- **Methodology:**
  - Defines constants for energy costs per bit transmitted (ETX) and received (ERX), and a fixed energy cost for data processing on the Valve (EC_VALVE_PROCESSING).
  - Defines constants for the number of sensor events (updates) per day and valve processing events per day.
  - Specifies the byte sizes for various CoAP and MQTT message types (GET, PUBLISH, ACK, CONNECT, etc.).
  - Calculates communication energy based on message size (converted to bits) and the ETX/ERX constants.
  - Calculates total energy for each device (Sensor, Valve) in each scenario (CoAP, MQTT) by summing:
  - One-time setup communication costs (like CoAP Observe setup or MQTT Connect/Subscribe).

- Periodic communication costs are based on daily event frequency.
- Periodic processing costs for the Valve (where applicable).

Python Script:

```
# Constants
ETX, ERX, EC_VALVE_PROCESSING = 50e-9, 58e-9, 2.4e-3 # J/bit, J/bit, J
SENSOR_EVENTS_PER_DAY = (24 * 60) / 5 # Sensor sends 288 updates per day (every 5 minutes)
VALVE_PROCESSING_EVENTS_PER_DAY = (24 * 60) / 30 # Valve processes data every 30 minutes (48 times a day)

# CoAP Sizes (Bytes)
coap_get_req_size, coap_get_resp_size, coap_empty_ack_size = 60, 55, 14

# MQTT Sizes (Bytes)
mqtt_subscribe_size, mqtt_suback_size, mqtt_publish_size = 58, 52, 68
mqtt_connect_size, mqtt_connack_size = 54, 47
mqtt_pingreq_size, mqtt_pingresp_size = 52, 48 # Needed only for EQ2 estimate

def calculate_comm_energy(bytes_tx, bytes_rx): # Returns energy in Joules
    return ((bytes_tx * 8 * ETX) + (bytes_rx * 8 * ERX))

# EQ1a: CoAP GET+OBSERVE(NON updates)
def calculate_coap_observe():
    # Valve sends a NON-Confirmable GET+OBSERVE to sensor
    valve_setup = calculate_comm_energy(coap_get_req_size, coap_get_resp_size) # Only once
    sensor_setup = calculate_comm_energy(coap_get_resp_size, coap_get_req_size) # Only once
    sensor_periodic = calculate_comm_energy(bytes_tx = coap_get_resp_size, bytes_rx = 0) *
SENSOR_EVENTS_PER_DAY # Sensor sends NON updates (288 times a day)
    valve_periodic_comm = calculate_comm_energy(bytes_tx = 0, bytes_rx = coap_get_resp_size) *
SENSOR_EVENTS_PER_DAY # Valve receives NON updates (288 times a day)
    valve_processing = EC_VALVE_PROCESSING * VALVE_PROCESSING_EVENTS_PER_DAY # Every 30 minutes valve
processes the data (48 times a day)
    total_sensor = sensor_setup + sensor_periodic # Joules
    total_valve = valve_setup + valve_periodic_comm + valve_processing # Joules
    return total_sensor, total_valve

# EQ1b: MQTT, QOS 0, no PINGs (Keep Alive > 5 min)
def calculate_mqtt_no_ping():
    sensor_connect = calculate_comm_energy(mqtt_connect_size, mqtt_connack_size) # Sensor connects to
broker once and receives CONNACK
    sensor_publish = calculate_comm_energy(mqtt_publish_size, 0) * SENSOR_EVENTS_PER_DAY # Sensor
publishes 288 times a day (QoS 0)
    valve_connect = calculate_comm_energy(mqtt_connect_size, mqtt_connack_size) # Valve connects to broker
once and receives CONNACK
    valve_subscribe = calculate_comm_energy(mqtt_subscribe_size, mqtt_suback_size) # Valve subscribes to
topic once and receives SUBACK
    valve_rx = calculate_comm_energy(0, mqtt_publish_size) * SENSOR_EVENTS_PER_DAY # Valve receives 288
messages a day (QoS 0)
    valve_processing = EC_VALVE_PROCESSING * VALVE_PROCESSING_EVENTS_PER_DAY # Every 30 minutes valve
processes the data (48 times a day)
    # Total energy calculations
    total_sensor = sensor_connect + sensor_publish # Joules
    total_valve = valve_connect + valve_subscribe + valve_rx + valve_processing # Joules
    return total_sensor, total_valve
```

Exercise Question 2:

Propose at least one solution for decreasing the energy consumption when passing using the Raspberry PI as a broker. Give a rough estimate of the energy saving that could be obtained with your solution: recompute the energy under your proposed configuration

ANSWER:

- **Optimized energy consumption:**
  - Sensor: 7.88 mJ (Unchanged)
  - Valve: 1.61 mJ (122.77 mJ saved)
  - Total = 9.48 mJ

The most effective way to reduce energy consumption for the battery-powered devices (Sensor and Valve) when using the Raspberry Pi is to **offload processing and aggregate data** on the Pi itself, since it is not power-constrained.

1. **Data Aggregation:** Instead of the Valve subscribing directly to the Sensor's frequent updates (every 5 minutes), the Raspberry Pi subscribes to these updates.
2. **Processing Offloading:** The Pi performs the necessary data processing/calculation that the Valve was previously doing every 30 minutes. Eliminating the 2.4 mJ processing cost that the valve was responsible for.
3. **Reduced Communication to Valve:** The Pi then publishes only the final processed result (e.g., an average, a status) to a *different* MQTT topic, specifically for the Valve. The Valve subscribes only to this new topic. This means the Valve now only receives messages when the processed result is ready (every 30 minutes), instead of receiving every raw sensor update (every 5 minutes).

Python Script

```
# EQ2: Optimizing MQTT by aggregating sensor data (5 min), and calculating the average on the raspberry pi which is unconstrained in power

# Offloading the processing to the raspberry pi (unconstrained in power) instead of the valve (constrained in power) --> eliminating the processing energy cost on the valve

# Sending the average to the valve every 30 minutes (48 times a day) instead of sending every 5 minutes (288 times a day) reducing the number of messages sent

def optimized_mqtt():
    # Sensor is unchaneged, but the valve now receives aggregated data every 30 minutes
    sensor_connect = calculate_comm_energy(mqtt_connect_size, mqtt_connack_size) # Sensor connects to broker once and receives CONNACK
    sensor_publish = calculate_comm_energy(mqtt_publish_size, 0) * SENSOR_EVENTS_PER_DAY # Sensor publishes 288 times a day (QoS 0)
    valve_connect = calculate_comm_energy(mqtt_connect_size, mqtt_connack_size) # Valve connects to broker once and receives CONNACK
    valve_subscribe = calculate_comm_energy(mqtt_subscribe_size, mqtt_suback_size) # Valve subscribes to topic once and receives SUBACK
    valve_rx = calculate_comm_energy(0, mqtt_publish_size) * VALVE_PROCESSING_EVENTS_PER_DAY # Valve receives 48 (previously 288) messages a day (QoS 0)
    # Total energy calculations
    total_sensor = sensor_connect + sensor_publish # Joules
    total_valve = valve_connect + valve_subscribe + valve_rx # Joules
    return total_sensor, total_valve
```